

JavaScript OOP, jQuery & AJAX

Study Notes — Day 4: OOP in JS, jQuery Framework, jQuery Events, jQuery UI Components, AJAX

1. Object-Oriented Programming (OOP) in JavaScript

OOP is a way of organizing code around "objects" — bundles of data (properties) and behavior (methods) — instead of writing everything as separate functions and variables. JavaScript supports OOP through object literals, constructor functions, prototypes, and the modern class syntax.

1.1 The Four Pillars of OOP

- Encapsulation — keeping data (properties) and the methods that work on that data together inside one object, and hiding internal details from outside code.
- Abstraction — exposing only what the user of an object needs (simple methods), while hiding the complicated logic behind them.
- Inheritance — one class can reuse and extend the properties/methods of another class, avoiding repeated code.
- Polymorphism — different classes can implement the same method name in their own way (e.g. every "shape" has an area() method, but each calculates it differently).

1.2 Objects and the Constructor Function (old style)

Before ES6 classes, objects were created using constructor functions with the 'new' keyword:

```
function Student(name, marks) {  
  this.name = name;  
  this.marks = marks;  
  this.showResult = function () {  
    console.log(this.name + ' scored ' + this.marks);  
  };  
}
```

```
const s1 = new Student('Riya', 88);  
s1.showResult(); // Riya scored 88
```

Note: 'this' inside the function refers to the new object being created.

1.3 ES6 Classes (modern style)

A class is a template for creating objects. It bundles properties (set in the constructor) and methods together.

```
class Student {  
  constructor(name, marks) {  
    this.name = name; // property  
    this.marks = marks; // property  
  }  
  showResult() { // method  
    console.log(`${this.name} scored ${this.marks}`);  
  }  
}  
  
const s1 = new Student('Riya', 88);
```

```
const s2 = new Student('Aman', 72);
s1.showResult(); // Riya scored 88
s2.showResult(); // Aman scored 72
```

1.4 Inheritance with 'extends' and 'super'

A child class can inherit properties and methods from a parent class using extends, and can call the parent's constructor using super().

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log(`Hi, I am ${this.name}`);
  }
}

class Student extends Person {
  constructor(name, course) {
    super(name); // calls Person's constructor
    this.course = course;
  }
  greet() { // polymorphism: overrides Person's greet()
    super.greet();
    console.log(`I am studying ${this.course}`);
  }
}

const st = new Student('Kiran', 'Full Stack Development');
st.greet();
// Hi, I am Kiran
// I am studying Full Stack Development
```

1.5 Encapsulation with Private Fields (#)

```
class BankAccount {
  #balance; // private property (# makes it inaccessible outside class)
  constructor(owner, balance) {
    this.owner = owner;
    this.#balance = balance;
  }
  deposit(amount) {
    this.#balance += amount;
  }
  getBalance() {
    return this.#balance;
  }
}

const acc = new BankAccount('Sam', 1000);
acc.deposit(500);
console.log(acc.getBalance()); // 1500
console.log(acc.#balance); // Error - not accessible
```

2. Linking JavaScript / jQuery with HTML

A JavaScript (or jQuery) file is connected to an HTML page with a `<script>` tag, either written inline or pointing to an external file with the `src` attribute. jQuery itself is just a JavaScript library, so it is linked the same way — either from a CDN link or a downloaded file — and must be loaded before your own script that uses it.

```
<!-- Load jQuery library first -->
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

<!-- Then your own script that uses jQuery -->
<script src="script.js"></script>
// script.js
$(document).ready(function () {
    console.log('Page is fully loaded, jQuery is ready to use');
});
```

Note: `$(document).ready()` makes sure your jQuery code only runs after the whole HTML page has loaded, so it never tries to select an element that doesn't exist yet.

3. jQuery Events

An event is something the user does on the page — for example clicking, typing, moving the mouse, or submitting a form. jQuery gives simple, ready-made methods to detect these events and run code in response.

3.1 `click()` and `dblclick()`

```
$('#btn').click(function () {
    alert('Button was clicked once');
});

$('#btn').dblclick(function () {
    alert('Button was double-clicked');
});
```

3.2 `mouseenter()` and `mouseleave()` / `hover()`

`mouseenter()` runs when the mouse comes onto an element; `mouseleave()` runs when the mouse goes away from it. `hover()` combines both in one method call.

```
$('#box').mouseenter(function () {
    $(this).css('background', 'yellow');
});
$('#box').mouseleave(function () {
    $(this).css('background', 'white');
});

// Same effect using hover(enterFn, leaveFn)
$('#box').hover(
    function () { $(this).css('background', 'yellow'); },
    function () { $(this).css('background', 'white'); }
);
```

3.3 keyup() — typing

keyup() runs when the user releases a key while typing, so it is commonly used for live search boxes or character counters.

```
$('#search').keyup(function () {  
    console.log('You typed: ' + $(this).val());  
});
```

3.4 focus() and blur()

```
$('#email').focus(function () {  
    $(this).css('border', '2px solid blue');    // clicked inside the input  
});  
$('#email').blur(function () {  
    $(this).css('border', '1px solid gray');    // left the input  
});
```

3.5 change() — dropdowns, checkboxes, radio buttons

```
$('#country').change(function () {  
    console.log('Selected: ' + $(this).val());  
});
```

3.6 submit() and preventDefault()

submit() runs when a form is sent. preventDefault() stops the page from reloading, which is essential when you want to validate or send form data with jQuery/AJAX instead of a normal page reload.

```
$('#myForm').submit(function (e) {  
    e.preventDefault();    // stop the page from reloading  
    console.log('Form submitted without reload');  
});
```

3.7 on() — the modern, recommended way

on('event', function) can attach any event, including events on elements added to the page later. It is the general-purpose replacement for click(), keyup(), etc.

```
$('#btn').on('click', function () {  
    console.log('Clicked using on()');  
});  
  
// Works even for elements added dynamically later:  
$('#list').on('click', 'li', function () {  
    console.log('You clicked: ' + $(this).text());  
});
```

3.8 The Event Object (e)

Whenever an event happens, jQuery passes an event object — usually named e — into the handler function. It carries useful information such as which key was pressed or which element triggered the event.

```
$('#input1').keyup(function (e) {  
    console.log('Key code:', e.keyCode);  
});
```

```
    console.log('Target element:', e.target);  
});
```

3.9 off() — removing an event

```
function sayHi() { alert('Hi!'); }  
$('#btn').on('click', sayHi);  
  
// later, stop listening for clicks:  
$('#btn').off('click', sayHi);
```

4. jQuery UI Components

jQuery UI is an add-on library built on top of jQuery. It provides ready-made, interactive widgets (calendars, sliders, popups, etc.) so you don't have to build them from scratch. It needs both the jQuery UI JS file and its CSS theme file linked in the page.

```
<link rel="stylesheet"  
href="https://code.jquery.com/ui/1.13.2/themes/base/jquery-ui.css">  
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>  
<script src="https://code.jquery.com/ui/1.13.2/jquery-ui.min.js"></script>
```

4.1 Datepicker — shows a calendar to pick a date

```
<input id="date" type="text">  
<script> $('#date').datepicker(); </script>
```

4.2 Accordion — sections that open and close on click

```
<div id="accordion">  
  <h3>Section 1</h3><div><p>Content 1</p></div>  
  <h3>Section 2</h3><div><p>Content 2</p></div>  
</div>  
<script> $('#accordion').accordion(); </script>
```

4.3 Tabs — shows content in tabs

```
<div id="tabs">  
  <ul><li><a href="#tab1">Tab 1</a></li><li><a href="#tab2">Tab  
2</a></li></ul>  
  <div id="tab1">First tab content</div>  
  <div id="tab2">Second tab content</div>  
</div>  
<script> $('#tabs').tabs(); </script>
```

4.4 Dialog — shows a popup box on the page

```
<div id="dialog" title="Message">This is a popup box.</div>  
<script> $('#dialog').dialog(); </script>
```

4.5 Autocomplete — shows suggestions while you type

```
<input id="city">
<script>
  const cities = ['Delhi', 'Mumbai', 'Chennai', 'Kolkata'];
  $('#city').autocomplete({ source: cities });
</script>
```

4.6 Slider — a bar you drag to pick a number

```
<div id="slider"></div>
<script> $('#slider').slider({ min: 0, max: 100, value: 50 }); </script>
```

4.7 Progressbar — shows how much work is done

```
<div id="progress"></div>
<script> $('#progress').progressbar({ value: 70 }); </script>
```

5. AJAX (Asynchronous JavaScript and XML)

AJAX is a technique used in web development to exchange data between a web page and a server without reloading the entire page. Despite the name, modern AJAX almost always exchanges JSON data rather than XML.

5.1 How AJAX Works

- 1. You click a button (or some event triggers the request).
- 2. A request is sent to the server in the background.
- 3. The server processes the request and sends data back.
- 4. The page updates with the new data — no reload.

5.2 Making an AJAX Request with jQuery

```
$('#loadBtn').click(function () {
  $.ajax({
    url: 'https://api.example.com/users',
    method: 'GET',
    success: function (data) {
      console.log('Data received:', data);
      $('#result').text(data.name); // update page without reload
    },
    error: function (err) {
      console.log('Request failed:', err);
    }
  });
});
```

jQuery also offers shorter helper methods for common cases:

```
$.get('https://api.example.com/users', function (data) {
  console.log(data);
});

$.post('https://api.example.com/users', { name: 'Sam' }, function (data) {
```

```
console.log('Saved:', data);  
});
```

5.3 JSON Text vs JSON Object

JSON Text	JSON Object
Data represented in JSON format as a string, e.g. <code>'{"name":"Sam"}'</code> .	A real JavaScript object in memory, e.g. <code>{ name: "Sam" }</code> , with properties you can access directly.
Used for sending/receiving data over a network — text travels well over HTTP.	Used inside your JavaScript code to actually read or change the data (<code>obj.name</code>).
Convert text → object with <code>JSON.parse(text)</code> .	Convert object → text with <code>JSON.stringify(obj)</code> before sending it.

```
const jsonText = '{"name":"Sam","age":21}'; // JSON text (a string)  
const jsonObj = JSON.parse(jsonText);      // now a real JS object  
console.log(jsonObj.name);                  // Sam  
  
const backToText = JSON.stringify(jsonObj); // JS object -> JSON text  
console.log(backToText);                    // '{"name":"Sam","age":21}'
```

Note: This document is prepared from personal handwritten class notes, expanded with runnable code examples for revision and practice.